

EDS_ASSIGNMENT_02

NAME: ABHIMANYU HARPALE

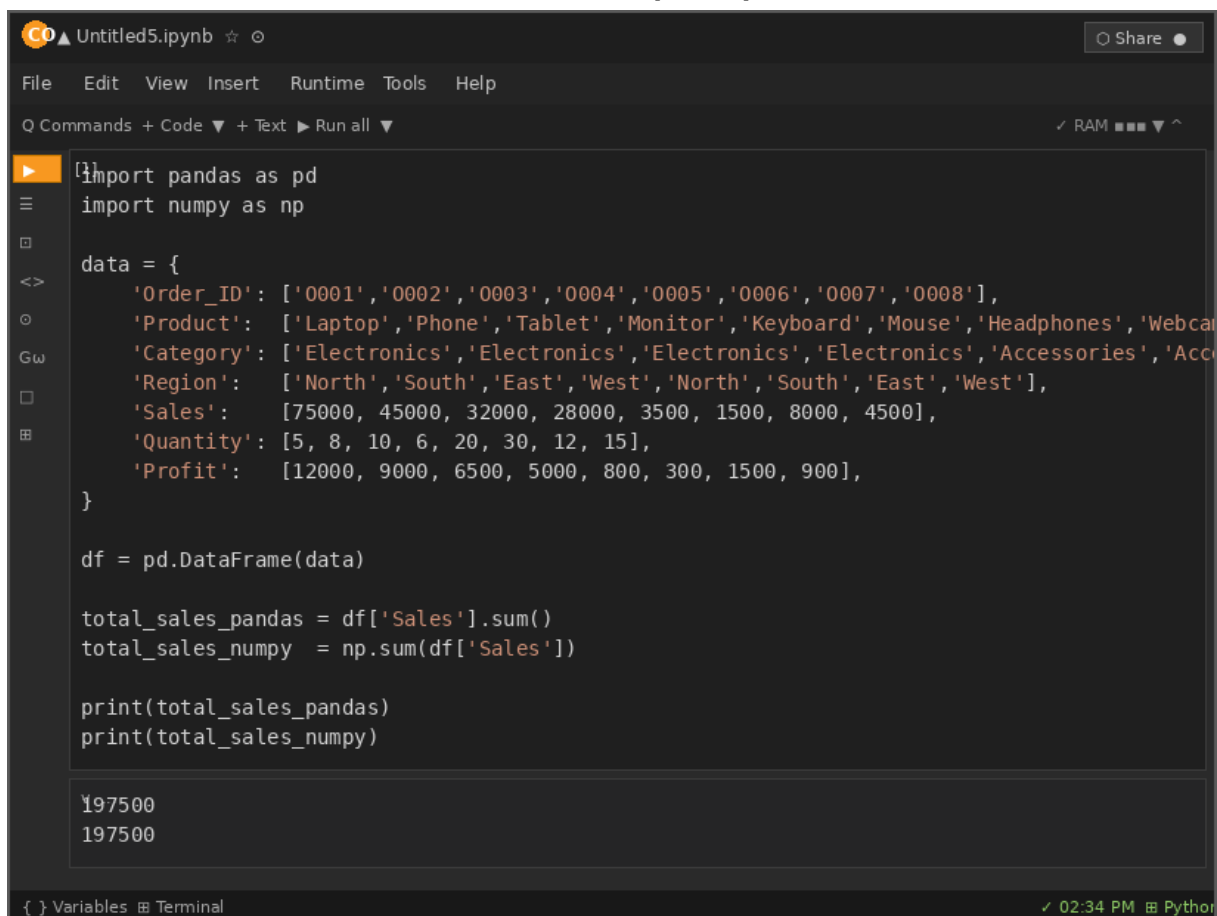
PRN: 202501070223

BATCH: ET 2

BRANCH: ENTC

DATASET: SALES

Problem Statement 1: Find the total revenue (Sales) from the Sales dataset.



```
import pandas as pd
import numpy as np

data = {
    'Order_ID': ['0001', '0002', '0003', '0004', '0005', '0006', '0007', '0008'],
    'Product': ['Laptop', 'Phone', 'Tablet', 'Monitor', 'Keyboard', 'Mouse', 'Headphones', 'Webcam'],
    'Category': ['Electronics', 'Electronics', 'Electronics', 'Electronics', 'Accessories', 'Accessories', 'Accessories', 'Accessories'],
    'Region': ['North', 'South', 'East', 'West', 'North', 'South', 'East', 'West'],
    'Sales': [75000, 45000, 32000, 28000, 3500, 1500, 8000, 4500],
    'Quantity': [5, 8, 10, 6, 20, 30, 12, 15],
    'Profit': [12000, 9000, 6500, 5000, 800, 300, 1500, 900],
}

df = pd.DataFrame(data)

total_sales_pandas = df['Sales'].sum()
total_sales_numpy = np.sum(df['Sales'])

print(total_sales_pandas)
print(total_sales_numpy)
```

197500
197500

Problem Statement 2: Find the top 3 highest-selling products.

Untitled5.ipynb ☆ ⓘ Share

File Edit View Insert Runtime Tools Help

Q Commands + Code ▼ + Text ▶ Run all ▼ ✓ RAM ███ ▼ ^

```
import pandas as pd

df = pd.DataFrame(data)

top_products = df.sort_values(by='Sales', ascending=False).head(3)
print(top_products[['Product', 'Sales']])
```

	Product	Sales
0	Laptop	75000
1	Phone	45000
2	Tablet	32000

{ } Variables ▢ Terminal ✓ 02:34 PM ▢ Python

Problem Statement 3: Find the product with the highest profit.

Untitled5.ipynb ☆ ⓘ Share

File Edit View Insert Runtime Tools Help

Q Commands + Code ▼ + Text ▶ Run all ▼ ✓ RAM ███ ▼ ^

```
import pandas as pd

df = pd.DataFrame(data)

max_profit_product = df[df['Profit'] == df['Profit'].max()]
print(max_profit_product[['Product', 'Profit']])
```

	Product	Profit
0	Laptop	12000

{ } Variables ▢ Terminal ✓ 02:34 PM ▢ Python

Problem Statement 4: Count the number of orders from each region.

Untitled5.ipynb ☆ ⓘ Share

File Edit View Insert Runtime Tools Help

Q Commands + Code ▼ + Text ▶ Run all ▼ ✓ RAM ███ ▼ ^

```
import pandas as pd

df = pd.DataFrame(data)

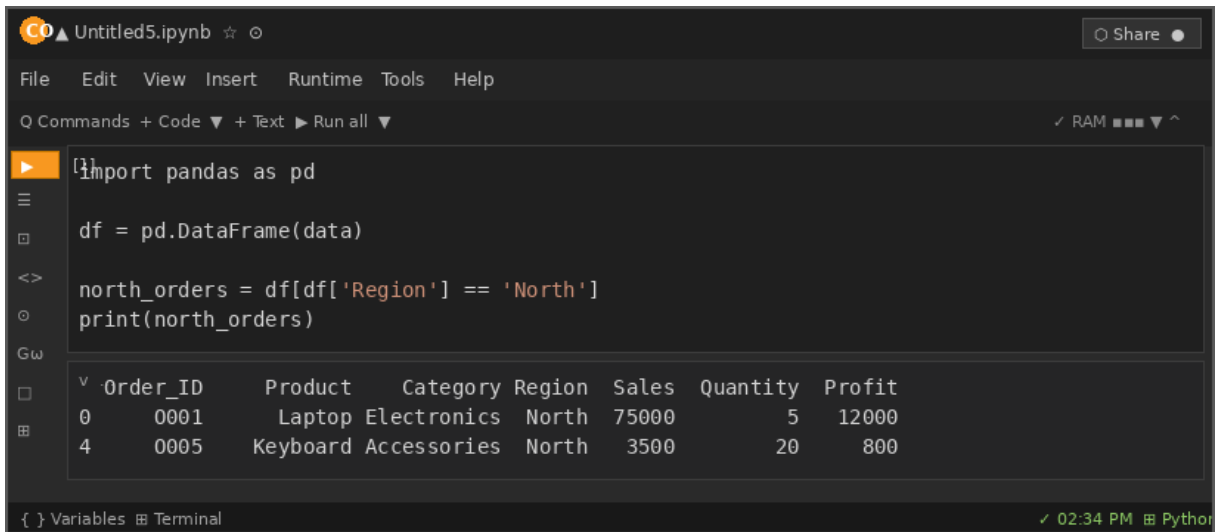
region_count = df['Region'].value_counts()
print(region_count)
```

Region	count
North	2
South	2
East	2
West	2

Name: count, dtype: int64

{ } Variables ▢ Terminal ✓ 02:34 PM ▢ Python

Problem Statement 5: Find all orders from the North region.

A screenshot of a Jupyter Notebook interface. The title bar shows 'Untitled5.ipynb' with a share button. The menu bar includes File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with 'Q Commands', '+ Code', '+ Text', and 'Run all'. The code cell contains the following Python code:

```
[1]: import pandas as pd

df = pd.DataFrame(data)

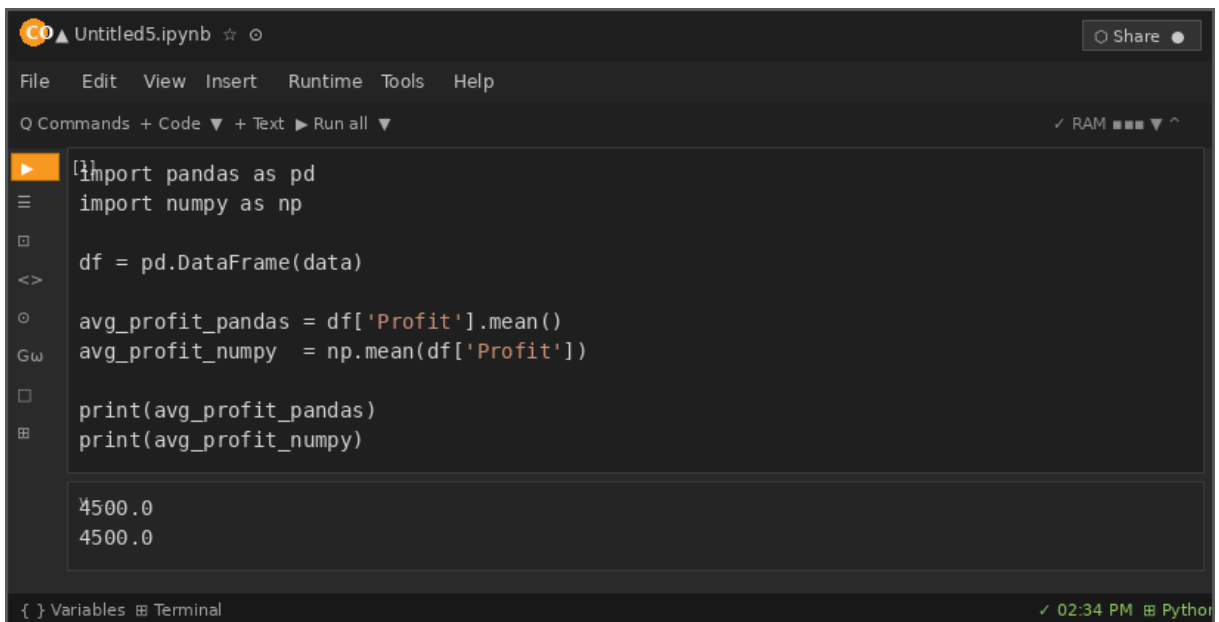
north_orders = df[df['Region'] == 'North']
print(north_orders)
```

The output cell displays a DataFrame with 2 rows and 7 columns: Order_ID, Product, Category, Region, Sales, Quantity, and Profit. The data is as follows:

Order_ID	Product	Category	Region	Sales	Quantity	Profit	
0	0001	Laptop	Electronics	North	75000	5	12000
4	0005	Keyboard	Accessories	North	3500	20	800

The bottom status bar shows '{ } Variables', 'Terminal', and a timestamp of '02:34 PM' with a Python logo.

Problem Statement 6: Find the average profit across all products.

A screenshot of a Jupyter Notebook interface. The title bar shows 'Untitled5.ipynb' with a share button. The menu bar includes File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with 'Q Commands', '+ Code', '+ Text', and 'Run all'. The code cell contains the following Python code:

```
[1]: import pandas as pd
import numpy as np

df = pd.DataFrame(data)

avg_profit_pandas = df['Profit'].mean()
avg_profit_numpy = np.mean(df['Profit'])

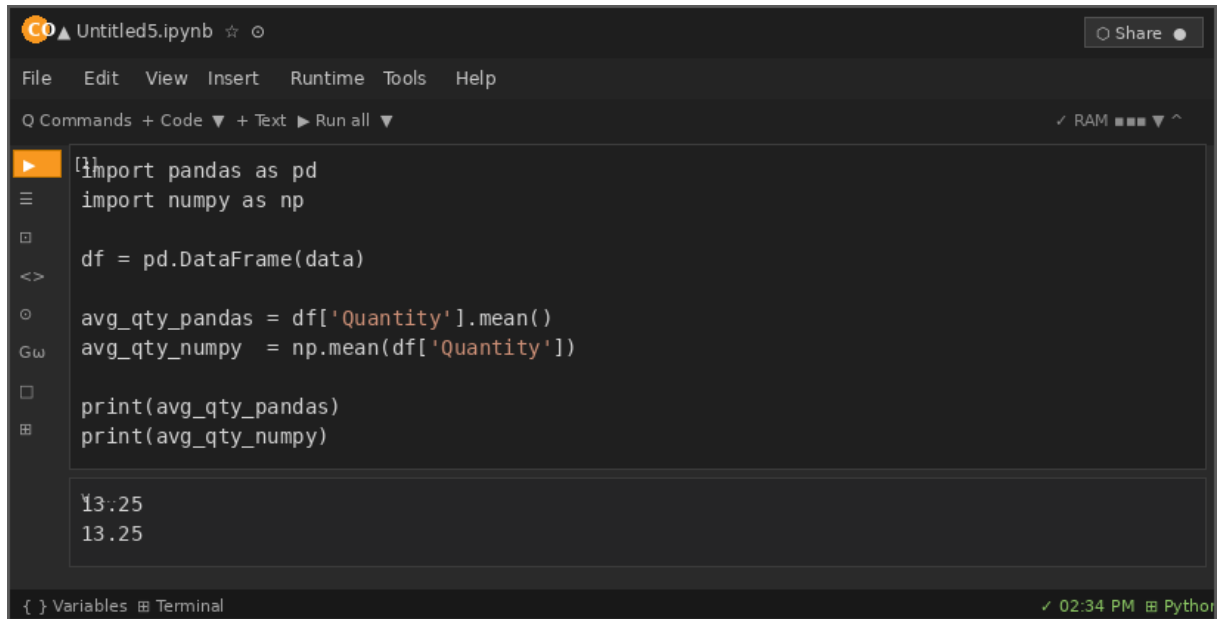
print(avg_profit_pandas)
print(avg_profit_numpy)
```

The output cell displays two lines of text:

```
4500.0
4500.0
```

The bottom status bar shows '{ } Variables', 'Terminal', and a timestamp of '02:34 PM' with a Python logo.

Problem Statement 7: Find the average quantity ordered per product.

A screenshot of a Jupyter Notebook interface. The top bar shows the file name 'Untitled5.ipynb' and a 'Share' button. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Q Commands', '+ Code', '+ Text', and 'Run all'. The code cell contains the following Python code:

```
import pandas as pd
import numpy as np

df = pd.DataFrame(data)

avg_qty_pandas = df['Quantity'].mean()
avg_qty_numpy = np.mean(df['Quantity'])

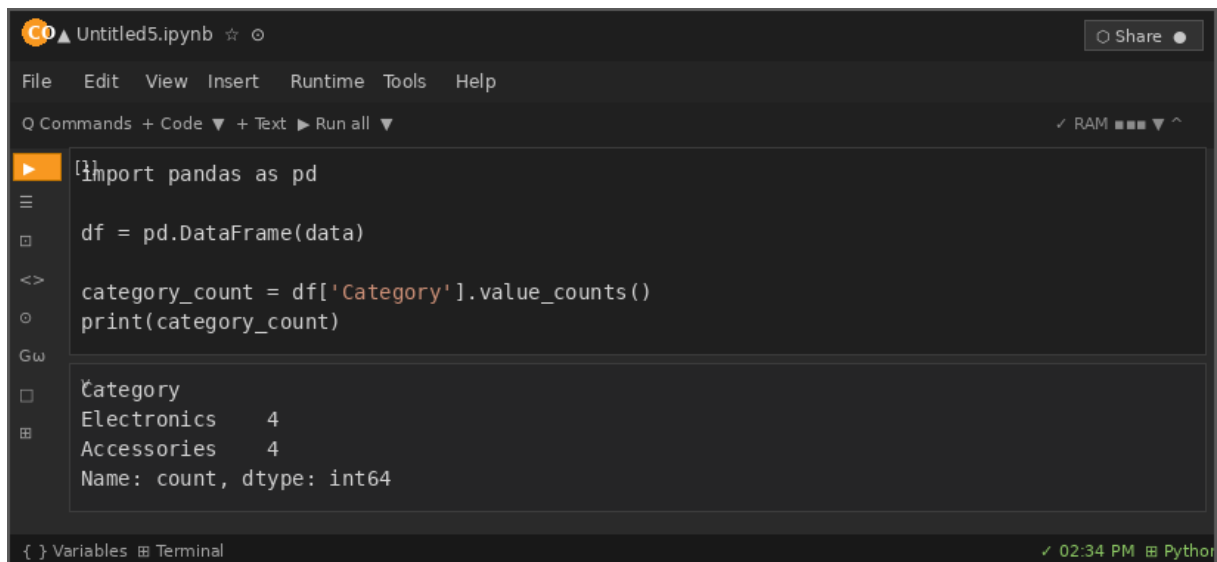
print(avg_qty_pandas)
print(avg_qty_numpy)
```

The output cell shows the results:

```
13.25
13.25
```

The bottom status bar shows '{ } Variables', 'Terminal', and a timestamp '02:34 PM' with 'Python'.

Problem Statement 8: Find the number of products in each category.

A screenshot of a Jupyter Notebook interface. The top bar shows the file name 'Untitled5.ipynb' and a 'Share' button. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Q Commands', '+ Code', '+ Text', and 'Run all'. The code cell contains the following Python code:

```
import pandas as pd

df = pd.DataFrame(data)

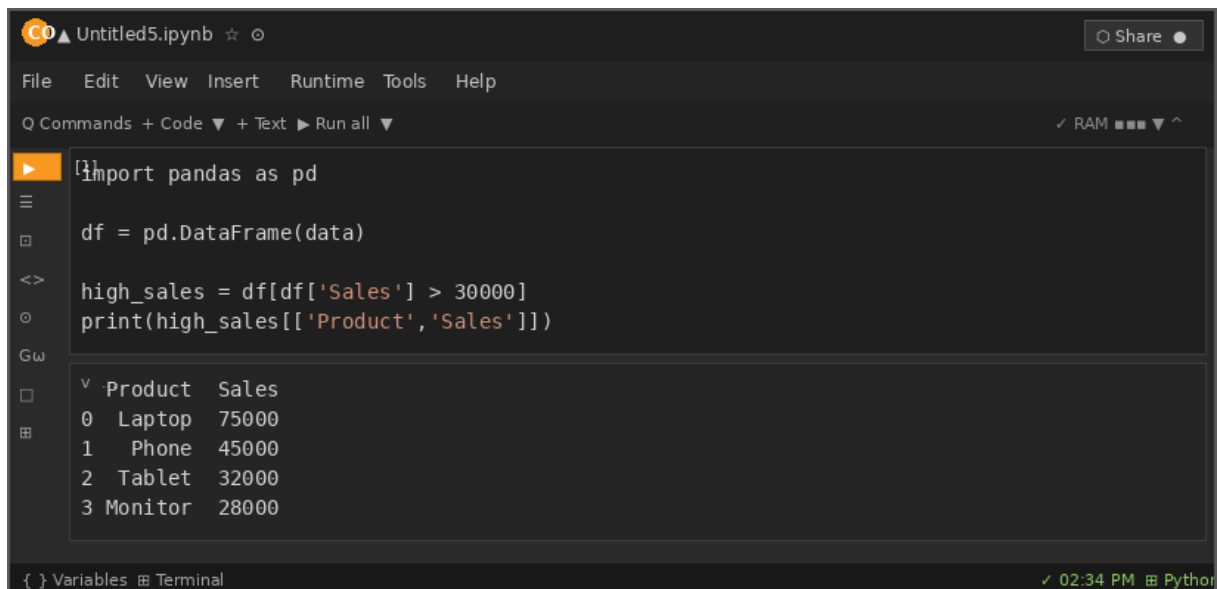
category_count = df['Category'].value_counts()
print(category_count)
```

The output cell shows the results:

```
Category
Electronics    4
Accessories    4
Name: count, dtype: int64
```

The bottom status bar shows '{ } Variables', 'Terminal', and a timestamp '02:34 PM' with 'Python'.

Problem Statement 9: Find all products with Sales greater than 30,000.

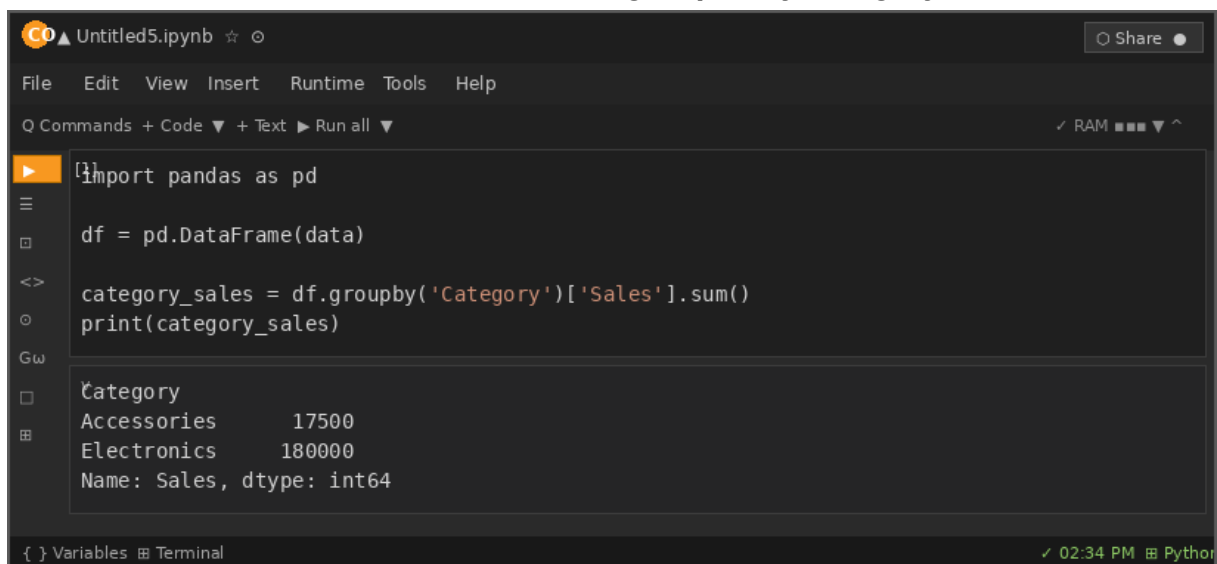


A Jupyter Notebook interface titled 'Untitled5.ipynb'. The code cell contains the following Python code: `import pandas as pd`, `df = pd.DataFrame(data)`, `high_sales = df[df['Sales'] > 30000]`, and `print(high_sales[['Product', 'Sales']])`. The output cell shows a DataFrame with two columns: 'Product' and 'Sales'. The data is as follows:

	Product	Sales
0	Laptop	75000
1	Phone	45000
2	Tablet	32000
3	Monitor	28000

The bottom status bar shows '02:34 PM Python'.

Problem Statement 10: Find the total sales grouped by category.

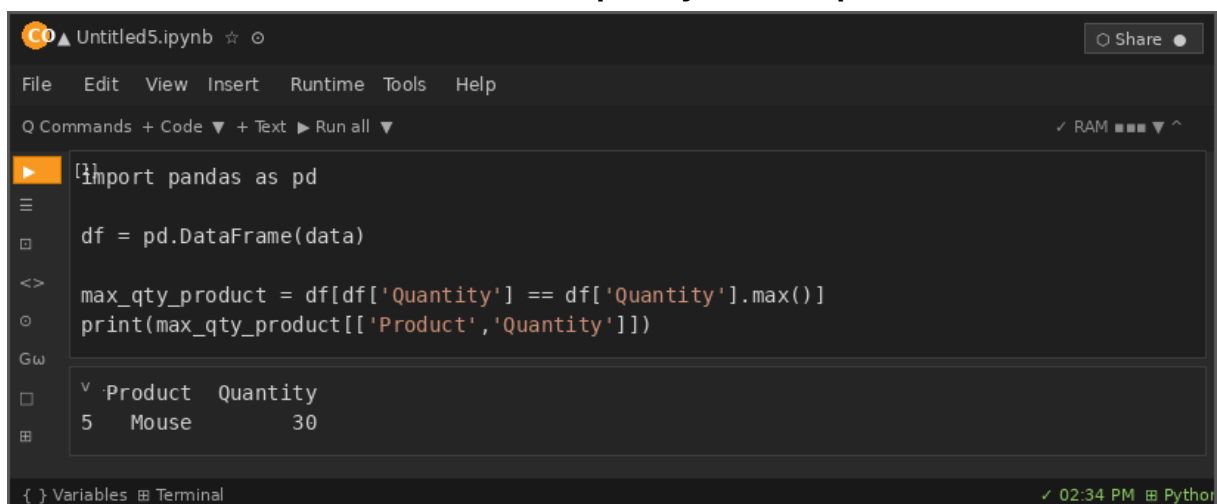


A Jupyter Notebook interface titled 'Untitled5.ipynb'. The code cell contains the following Python code: `import pandas as pd`, `df = pd.DataFrame(data)`, `category_sales = df.groupby('Category')['Sales'].sum()`, and `print(category_sales)`. The output cell shows a DataFrame with two columns: 'Category' and 'Sales'. The data is as follows:

Category	Sales
Accessories	17500
Electronics	180000

The bottom status bar shows '02:34 PM Python'.

Problem Statement 11: Find the most frequently ordered product.

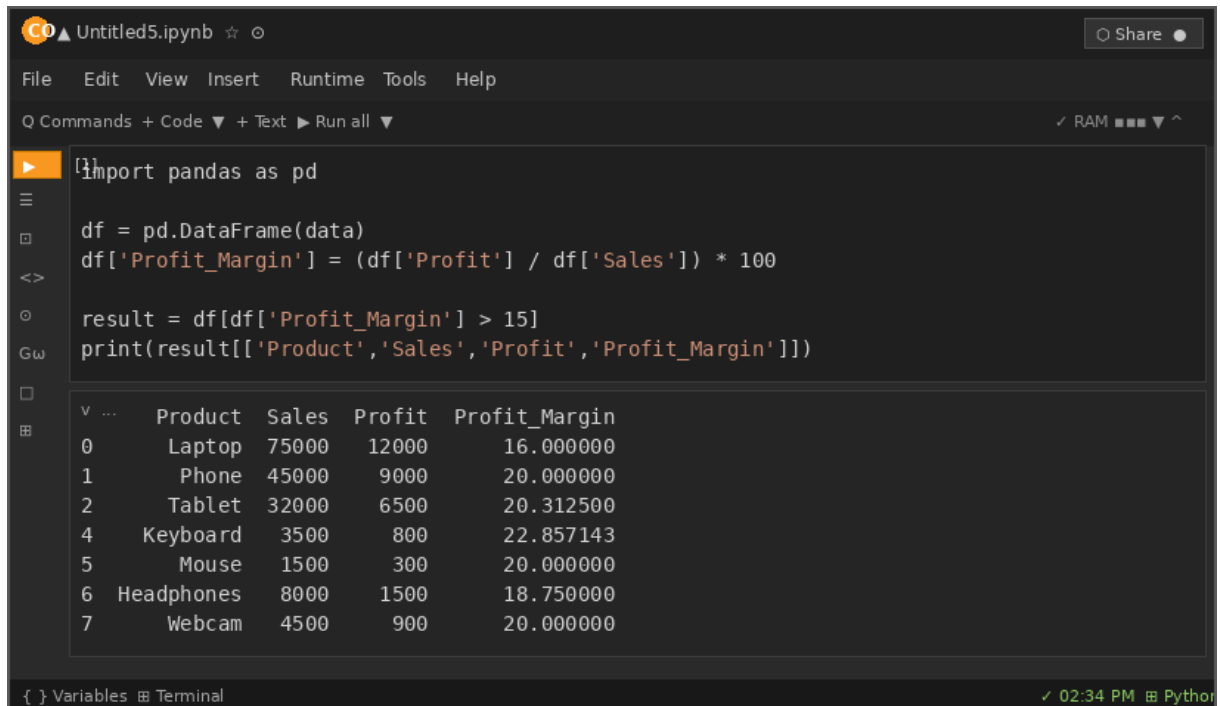


A Jupyter Notebook interface titled 'Untitled5.ipynb'. The code cell contains the following Python code: `import pandas as pd`, `df = pd.DataFrame(data)`, `max_qty_product = df[df['Quantity'] == df['Quantity'].max()]`, and `print(max_qty_product[['Product', 'Quantity']])`. The output cell shows a DataFrame with two columns: 'Product' and 'Quantity'. The data is as follows:

Product	Quantity
5 Mouse	30

The bottom status bar shows '02:34 PM Python'.

Problem Statement 12: Find products with profit margin greater than 15% of sales.



A Jupyter Notebook interface showing a Python script to calculate profit margins. The code imports pandas, creates a DataFrame from 'data', calculates 'Profit_Margin' as (Profit / Sales) * 100, filters for values greater than 15, and prints the results. The output is a table with 8 rows of product data.

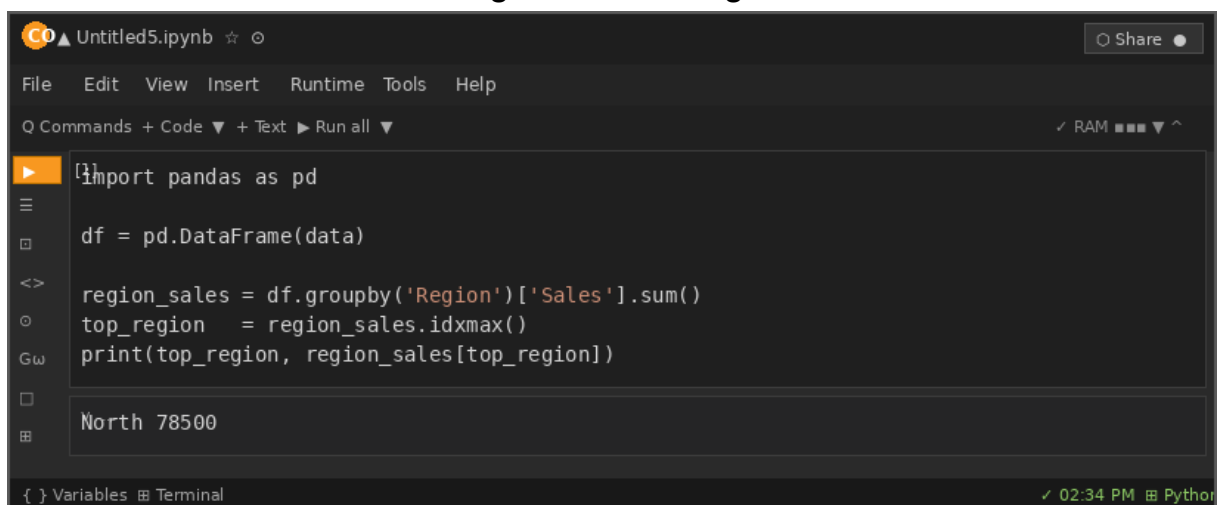
```
[1] import pandas as pd

df = pd.DataFrame(data)
df['Profit_Margin'] = (df['Profit'] / df['Sales']) * 100

result = df[df['Profit_Margin'] > 15]
print(result[['Product', 'Sales', 'Profit', 'Profit_Margin']])
```

	Product	Sales	Profit	Profit_Margin
0	Laptop	75000	12000	16.000000
1	Phone	45000	9000	20.000000
2	Tablet	32000	6500	20.312500
4	Keyboard	3500	800	22.857143
5	Mouse	1500	300	20.000000
6	Headphones	8000	1500	18.750000
7	Webcam	4500	900	20.000000

Problem Statement 13: Find the region with the highest total sales.



A Jupyter Notebook interface showing a Python script to find the region with the highest total sales. The code groups sales by region, finds the maximum, and prints the result. The output is 'North 78500'.

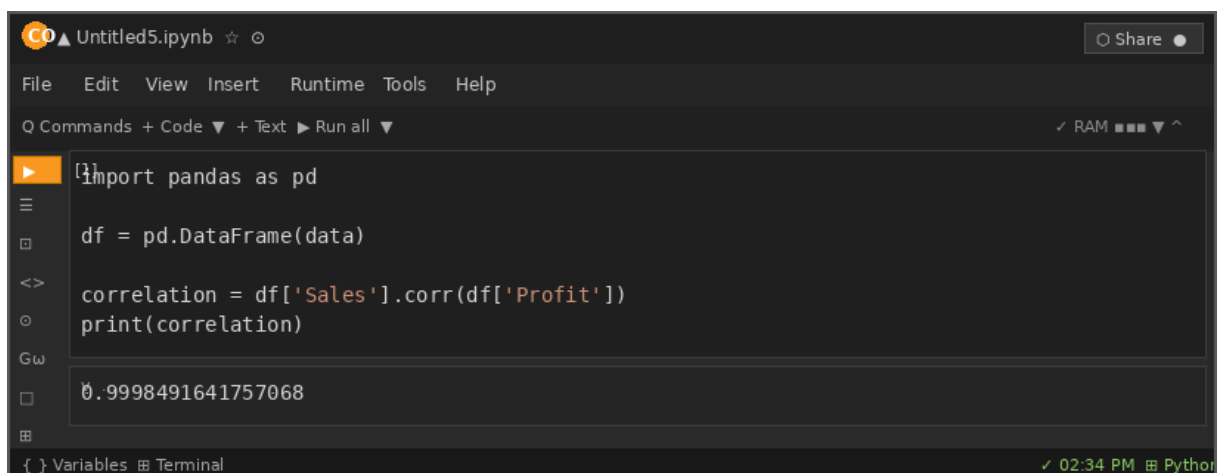
```
[1] import pandas as pd

df = pd.DataFrame(data)

region_sales = df.groupby('Region')['Sales'].sum()
top_region = region_sales.idxmax()
print(top_region, region_sales[top_region])
```

North 78500

Problem Statement 14: Find the correlation between Sales and Profit.



A Jupyter Notebook interface showing a Python script to calculate the correlation between Sales and Profit. The code uses the .corr() method on the DataFrame. The output is 0.9998491641757068.

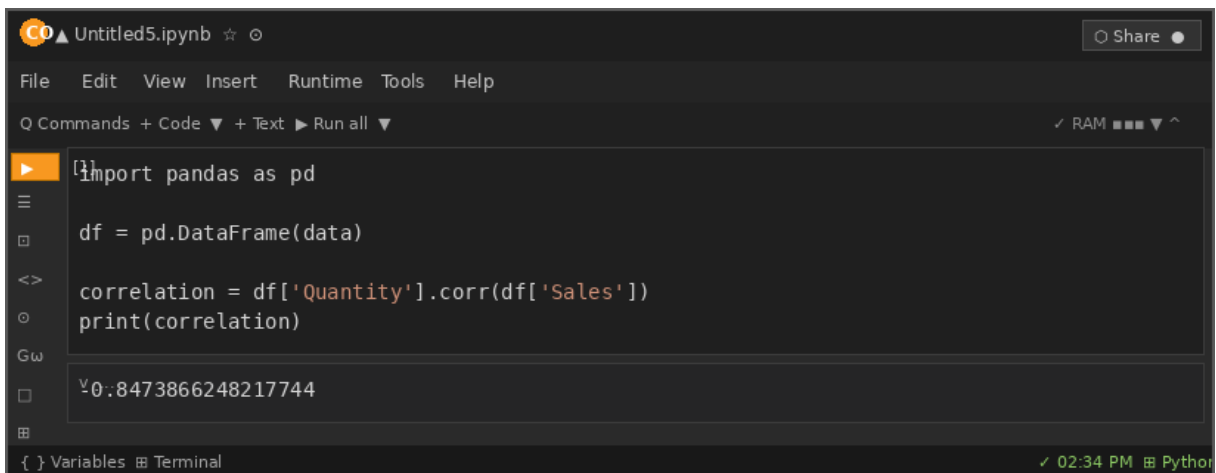
```
[1] import pandas as pd

df = pd.DataFrame(data)

correlation = df['Sales'].corr(df['Profit'])
print(correlation)
```

0.9998491641757068

Problem Statement 15: Analyze the relationship between Quantity and Sales.

A screenshot of a Jupyter Notebook interface. The title bar shows 'Untitled5.ipynb' with a share button. The menu bar includes File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with 'Q Commands', '+ Code', '+ Text', 'Run all', and a RAM usage indicator. The code cell contains the following Python code:

```
[1]: import pandas as pd

df = pd.DataFrame(data)

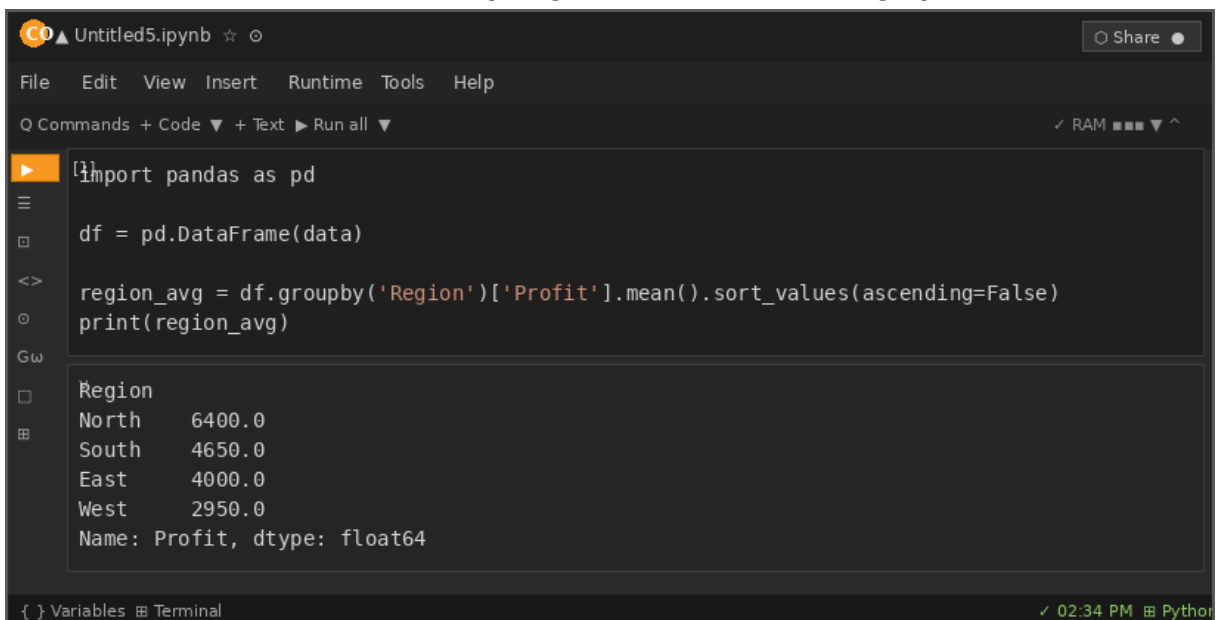
correlation = df['Quantity'].corr(df['Sales'])
print(correlation)
```

The output cell shows the correlation value:

```
0.8473866248217744
```

 The bottom status bar shows '{ } Variables', 'Terminal', a green checkmark, '02:34 PM', and 'Python'.

Problem Statement 16: Find the top regions based on average profit.

A screenshot of a Jupyter Notebook interface. The title bar shows 'Untitled5.ipynb' with a share button. The menu bar includes File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with 'Q Commands', '+ Code', '+ Text', 'Run all', and a RAM usage indicator. The code cell contains the following Python code:

```
[1]: import pandas as pd

df = pd.DataFrame(data)

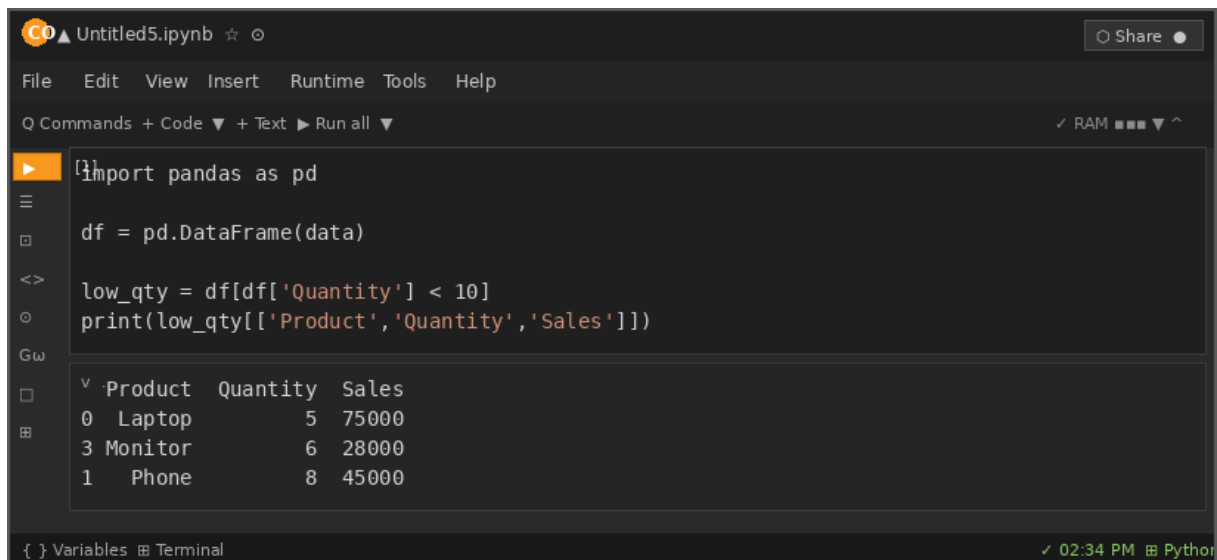
region_avg = df.groupby('Region')['Profit'].mean().sort_values(ascending=False)
print(region_avg)
```

The output cell shows the average profit for each region:

```
Region
North    6400.0
South    4650.0
East     4000.0
West     2950.0
Name: Profit, dtype: float64
```

 The bottom status bar shows '{ } Variables', 'Terminal', a green checkmark, '02:34 PM', and 'Python'.

Problem Statement 17: Find all products with quantity ordered below 10.



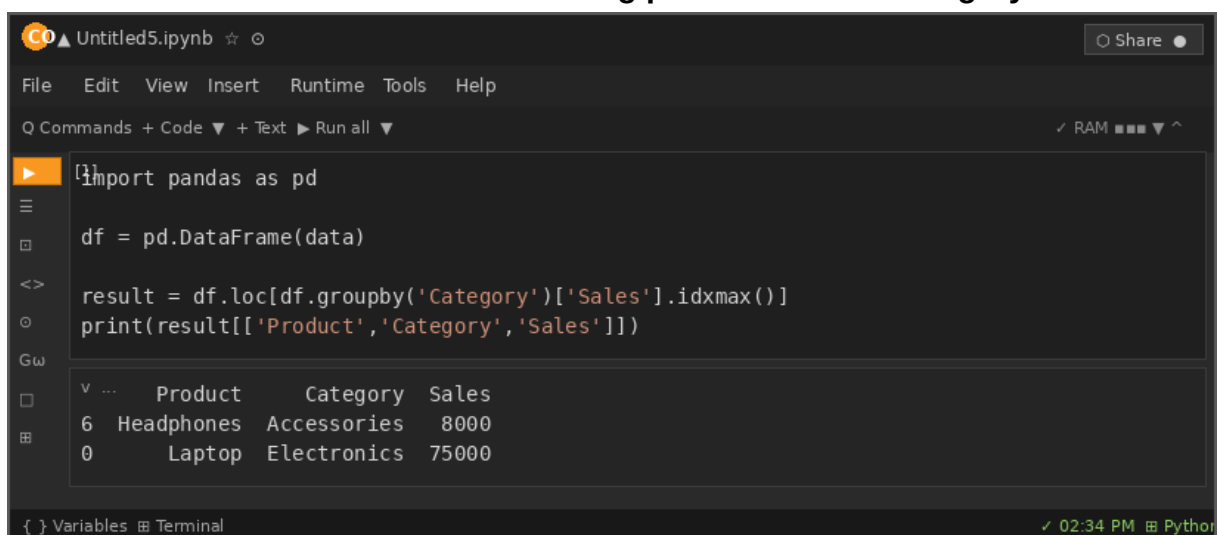
```
import pandas as pd

df = pd.DataFrame(data)

low_qty = df[df['Quantity'] < 10]
print(low_qty[['Product', 'Quantity', 'Sales']])
```

	Product	Quantity	Sales
0	Laptop	5	75000
3	Monitor	6	28000
1	Phone	8	45000

Problem Statement 18: Find the best-selling product in each category.



```
import pandas as pd

df = pd.DataFrame(data)

result = df.loc[df.groupby('Category')['Sales'].idxmax()]
print(result[['Product', 'Category', 'Sales']])
```

	Product	Category	Sales
6	Headphones	Accessories	8000
0	Laptop	Electronics	75000

Problem Statement 19: Show the distribution of sales quantities.

The image shows a Jupyter Notebook window titled 'Untitled5.ipynb'. The code cell contains the following Python code:

```
import pandas as pd

df = pd.DataFrame(data)

qty_distribution = df['Quantity'].value_counts().sort_index()
print(qty_distribution)
```

The output of the code is displayed below the code cell:

```
Quantity
5      1
6      1
8      1
10     1
12     1
15     1
20     1
30     1
Name: count, dtype: int64
```

The bottom status bar shows '02:34 PM Python'.

Problem Statement 20: Generate summary statistics of the Sales dataset.

The image shows a Jupyter Notebook window titled 'Untitled5.ipynb'. The code cell contains the following Python code:

```
import pandas as pd

df = pd.DataFrame(data)

print(df.describe())
```

The output of the code is displayed below the code cell:

```
v ...      Sales  Quantity  Profit
count      8.000000   8.000000   8.000000
mean    24687.500000  13.250000  4500.000000
std     24567.614107   8.523499  4001.785742
min      1500.000000   5.000000   300.000000
25%      3875.000000   6.500000   725.000000
50%     18250.000000  11.000000  2200.000000
75%     38250.000000  18.750000  7625.000000
max      75000.000000  30.000000 12000.000000
```

The bottom status bar shows '02:34 PM Python'.